

Enci/ VENTAS

Manual de Instalación Local

MVP Web Avanzado - Backend API, Frontend Web y App Vendedor

Versión 1.0

Julio 2026

Aplicación de Automatización de Fuerza de Ventas

Cliente	Enci / Enci Ventas
Representante receptor	Max
Equipo desarrollador	Grupo PLM - Universidad Tecnológica Metropolitana (UTEM)
Líder de equipo	Joshua Flores
Integrantes	Falon Berrios, Caroline Palacios y Eliseo Poblete
Repositorio oficial del MVP web	Chumajer14/Encipharm
Tipo de documento	Guía formal de instalación, configuración y validación local

BACKEND FastAPI / Python / uv	FRONTEND React + Vite	FIREBASE Auth + Admin SDK	LOCAL 127.0.0.1
----------------------------------	--------------------------	------------------------------	--------------------

Alcance de este manual

Este documento describe la preparación y ejecución local del monorepo del MVP web avanzado. La app Flutter nativa Android se entiende como entregable parcial externo al repositorio y no se instala desde este manual; su validación se realiza mediante paquete o APK provisto por el equipo, según el estado acordado con Max.

Tabla de Contenidos

1. Introducción
2. Alcance de instalación local
3. Requisitos previos
4. Preparación del repositorio
5. Configuración Firebase local
6. Variables de entorno
7. Instalación y ejecución del backend
8. Instalación y ejecución del frontend web
9. Ejecución de app vendedor web/PWA
10. Inicio rápido en Windows
11. Validaciones locales
12. Puertos, rutas y consola de pruebas
13. Problemas comunes y acciones correctivas
14. Criterio de instalación exitosa

1. Introducción

El presente manual establece el procedimiento técnico para instalar y ejecutar localmente el sistema Enci/ Ventas en un ambiente de desarrollo o validación controlada. Su objetivo es permitir que un responsable técnico pueda levantar el backend FastAPI, el frontend web React/Vite, la app vendedor web/PWA incluida en el repositorio y la consola de pruebas Swagger sin depender de un despliegue productivo.

La instalación local permite revisar flujos funcionales, ejecutar pruebas básicas, validar configuración de Firebase, comprobar la comunicación entre frontend y backend, revisar la autorización por token Bearer, verificar readiness y preparar evidencias técnicas antes de una instancia UAT o de salida controlada.

Advertencia operativa

La instalación local no equivale a ambiente productivo. Las credenciales, dominios, usuarios reales, seguridad perimetral, certificados, despliegue en Cloud Run/Vercel y validación UAT deben tratarse como actividades separadas de go-live.

2. Alcance de instalación local

El procedimiento cubre los componentes incluidos en el monorepo oficial del MVP web y sus servicios asociados. No cubre publicación productiva, configuración definitiva de dominios, entrega de credenciales reales al cliente, migraciones históricas de datos, pruebas de carga ni soporte evolutivo posterior.

Componente	Incluido en este manual	Observación
Backend API	Sí	Ejecución local de FastAPI con uvicorn, Firebase Admin y variables de entorno.
Frontend web	Sí	Ejecución local de React/Vite en puerto 5173.
App vendedor web/PWA	Sí	Ejecución local desde carpeta vendedor-app en puerto 5174 cuando aplique.
Firebase Auth	Sí	Requiere proyecto Firebase y Google Sign-In habilitado.
Firestore	Sí	Requiere proyecto configurado y credenciales válidas para backend.
IA RAG	Parcial	Puede ejecutarse si existen variables DeepSeek, almacenamiento documental y dependencias instaladas.
Flutter nativo Android	No	Entregable parcial externo al monorepo; se valida mediante APK o paquete independiente.
Go-live productivo	No	Se documenta en manual de despliegue y runbook operativo.

3. Requisitos previos

Antes de ejecutar el sistema se debe preparar una estación local con permisos de instalación, acceso al repositorio, herramientas de desarrollo y credenciales no productivas o autorizadas para pruebas.

Requisito	Versión o condición	Uso
Sistema operativo	Windows 10/11 recomendado	Permite uso directo del script iniciar-enci-local.bat.
Git	Instalado y disponible en PATH	Clonar o actualizar el repositorio.

Requisito	Versión o condición	Uso
Python	3.12 o superior	Ejecutar backend FastAPI y pruebas backend.
uv	Instalado y disponible en PATH	Resolver dependencias backend de forma reproducible.
Node.js	20 o superior recomendado	Instalar y ejecutar frontend web y app vendedor.
npm	Incluido con Node.js	Instalación de dependencias JavaScript.
Firebase	Proyecto activo	Autenticación Google, Firestore y Admin SDK.
Credenciales Admin SDK	Archivo local o variable JSON	Permiten al backend validar tokens y acceder a Firestore.
Navegador	Chrome, Edge o equivalente	Validación de login, frontend y consola Swagger.

3.1 Verificación rápida de herramientas

```
git --version
python --version
uv --version
node --version
npm --version
```

Si cualquiera de los comandos anteriores no responde, debe instalarse la herramienta correspondiente o corregirse la variable PATH antes de continuar.

4. Preparación del repositorio

La instalación debe realizarse desde una copia local limpia del repositorio oficial del MVP web. Se recomienda trabajar desde una ruta sin espacios ni caracteres especiales para evitar problemas de rutas en scripts, entornos virtuales o dependencias de Node.js.

4.1 Clonar repositorio

```
git clone https://github.com/Chumajer14/Encipharm.git
cd Encipharm
```

4.2 Actualizar copia existente

```
cd Encipharm
git status
git pull origin main
```

4.3 Estructura esperada

Ruta	Función
Backend/	API FastAPI, modelos, routers, servicios, configuración, pruebas y dependencias Python.
frontend/	Aplicación web React/Vite para el MVP comercial.
vendedor-app/	App vendedor web/PWA basada en Vite, independiente del Flutter externo.
docs/	Documentación técnica del proyecto.
iniciar-enci-local.bat	Lanzador Windows para levantar backend, frontend web y app vendedor local.
README.md	Resumen técnico principal del repositorio.

5. Configuración Firebase local

Los flujos autenticados dependen de Firebase Auth y de la validación backend mediante Firebase Admin SDK. Por ello, antes de iniciar el sistema se debe contar con un proyecto Firebase habilitado para pruebas, Google Sign-In activo y dominios locales autorizados.

5.1 Activar Google Sign-In

- 1 Ingresar a Firebase Console con una cuenta autorizada.
- 2 Abrir el proyecto Firebase asociado al ambiente de prueba.
- 3 Ir a Authentication > Sign-in method.
- 4 Habilitar el proveedor Google.
- 5 Configurar correo de soporte del proyecto si Firebase lo solicita.
- 6 Guardar los cambios y confirmar que el proveedor queda activo.

5.2 Autorizar dominios locales

En Authentication > Settings > Authorized domains deben existir, como mínimo, los siguientes dominios para pruebas locales:

- localhost
- 127.0.0.1

5.3 Obtener credenciales Admin SDK

El backend requiere credenciales de servicio para validar tokens y operar Firestore. Se admiten dos métodos, ambos fuera del repositorio:

Método	Variable	Descripción
Archivo local	GOOGLE_APPLICATION_CREDENTIALS	Ruta absoluta o relativa al archivo JSON de service account no versionado.
JSON serializado	FIREBASE_SERVICE_ACCOUNT_JSON	Contenido JSON completo guardado como variable de entorno segura.

Control de seguridad

Las credenciales Firebase Admin, archivos .env reales, tokens y llaves de proveedor nunca deben subirse al repositorio ni compartirse en canales no autorizados.

6. Variables de entorno

Cada componente requiere variables locales. El procedimiento recomendado es crear archivos .env a partir de los ejemplos del repositorio o de la configuración provista por el responsable técnico. Las variables deben ajustarse al proyecto Firebase autorizado.

6.1 Backend - Backend/.env

```
APP_NAME=Enci Ventas API
APP_ENV=development
APP_VERSION=1.0.0
CORS_ORIGINS=http://localhost:3000,http://localhost:5173,http://localhost:5174,http://127.0.0.1:5173,http://127.0.0.1:5174
FIREBASE_PROJECT_ID=your-project-id
GOOGLE_APPLICATION_CREDENTIALS=serviceAccountKey.json
```

6.2 Variables opcionales para consola /docs

```
FIREBASE_WEB_API_KEY=your-firebase-api-key
FIREBASE_WEB_AUTH_DOMAIN=your-auth-domain
FIREBASE_WEB_STORAGE_BUCKET=your-storage-bucket
FIREBASE_WEB_MESSAGING_SENDER_ID=your-sender-id
FIREBASE_WEB_APP_ID=your-app-id
FIREBASE_WEB_MEASUREMENT_ID=your-measurement-id
```

6.3 Variables RAG opcionales

```
DEEPSEEK_API_KEY=your-deepseek-api-key
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=deepseek-chat
CHROMA_PERSIST_DIR=./chroma_db
LOCAL_DOCUMENT_STORAGE_DIR=./document_storage
RAG_CHAT_RATE_LIMIT_PER_MINUTE=30
RAG_MAX_UPLOAD_BYTES=20971520
```

6.4 Frontend web - frontend/.env

```
VITE_APP_NAME=Enci Ventas
VITE_API_BASE_URL=http://127.0.0.1:8000
VITE_FIREBASE_API_KEY=your-firebase-api-key
VITE_FIREBASE_AUTH_DOMAIN=your-auth-domain
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-storage-bucket
VITE_FIREBASE_MESSAGING_SENDER_ID=your-sender-id
VITE_FIREBASE_APP_ID=your-app-id
VITE_FIREBASE_MEASUREMENT_ID=your-measurement-id
VITE_FIREBASE_FREE_TIER_MODE=true
```

6.5 App vendedor web/PWA - vendedor-app/.env

```
VITE_API_BASE_URL=http://127.0.0.1:8000
VITE_FIREBASE_API_KEY=your-firebase-api-key
VITE_FIREBASE_AUTH_DOMAIN=your-auth-domain
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-storage-bucket
VITE_FIREBASE_MESSAGING_SENDER_ID=your-sender-id
VITE_FIREBASE_APP_ID=your-app-id
```

7. Instalación y ejecución del backend

El backend se ejecuta desde la carpeta Backend mediante uv. La primera ejecución debe sincronizar dependencias, crear entorno local y validar que las variables de entorno estén disponibles.

7.1 Instalar dependencias

```
cd Backend
uv sync
```

7.2 Ejecutar API local

```
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

7.3 Verificar salud y readiness

```
Invoke-WebRequest http://127.0.0.1:8000/health
Invoke-WebRequest http://127.0.0.1:8000/readiness
```

Respuesta esperada para health:

```
{"status":"ok"}
```

El endpoint readiness debe devolver un reporte de validación. Si existen variables faltantes o placeholders, el estado puede advertir que el ambiente aún no está listo para UAT/go-live; eso no impide la revisión local, pero debe corregirse antes de validación formal.

8. Instalación y ejecución del frontend web

El frontend web corresponde a la aplicación principal de operación comercial. Consume la API FastAPI mediante token Bearer Firebase y debe ejecutarse apuntando al backend local.

8.1 Instalar dependencias

```
cd frontend  
npm install
```

8.2 Ejecutar frontend

```
npm run dev -- --host 127.0.0.1 --port 5173
```

8.3 Validación visual inicial

Abrir el navegador en <http://127.0.0.1:5173>. La pantalla de login debe cargar, permitir inicio de sesión con Google y redirigir al sistema según el rol del usuario.

9. Ejecución de app vendedor web/PWA

El repositorio incluye una app vendedor web/PWA independiente de la app Flutter nativa. Esta app permite validar en local flujos móviles web como cotizaciones, pipeline, forecast y Enci Chat desde una interfaz separada.

9.1 Instalar dependencias

```
cd vendedor-app  
npm install
```

9.2 Ejecutar app vendedor

```
npm run dev -- --host 127.0.0.1 --port 5174
```

Diferencia con Flutter

La carpeta vendedor-app no corresponde al entregable Flutter nativo Android. La app Flutter parcial fue desarrollada fuera del monorepo y debe tratarse como entregable externo validable por APK o paquete separado.

10. Inicio rápido en Windows

Cuando las dependencias ya están instaladas y los archivos .env configurados, el repositorio permite levantar los servicios locales mediante el script iniciar-enci-local.bat desde la raíz del proyecto.

```
.\iniciar-enci-local.bat
```

El script valida que uv esté disponible, comprueba dependencias de frontend y vendedor-app, inicia tres ventanas de terminal y abre las aplicaciones en el navegador.

Proceso	Comando interno	URL esperada
Backend	uv run uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload	http://127.0.0.1:8000
Frontend web	npm run dev -- --host 127.0.0.1 --port 5173	http://127.0.0.1:5173
App vendedor	npm run dev -- --host 127.0.0.1 --port 5174	http://127.0.0.1:5174

11. Validaciones locales

La instalación no debe considerarse correcta solamente porque los procesos inician. Deben ejecutarse validaciones técnicas mínimas para comprobar compilación, lint, pruebas y flujos autenticados.

11.1 Validación backend

```
cd Backend
uv run pytest
```

11.2 Validación frontend

```
cd frontend
npm run lint
npm run build
```

11.3 Validación app vendedor web/PWA

```
cd vendedor-app
npm run lint
npm run build
```

11.4 Checklist funcional mínimo

Validación	Resultado esperado
Backend /health	Responde status ok.
Backend /readiness	Entrega reporte de configuración sin exponer secretos.
Frontend carga	Login visible en http://127.0.0.1:5173.
Login Google	Popup abre, autentica y retorna token Firebase.
/auth/login	Sincroniza usuario autenticado en Firestore.
/me	Retorna uid, email, rol y activo.
/clientes	Responde según rol del usuario.
Dashboard	Carga métricas sin error crítico de API.
Swagger	Permite autorizar token y probar endpoints protegidos en ambiente no productivo.

12. Puertos, rutas y consola de pruebas

Servicio	Puerto	URL
Backend FastAPI	8000	http://127.0.0.1:8000
Frontend web	5173	http://127.0.0.1:5173
App vendedor web/PWA	5174	http://127.0.0.1:5174
Frontend alternativo	5175	Usado si el puerto principal está ocupado.
Swagger controlado	8000	http://127.0.0.1:8000/docs
OpenAPI JSON	8000	http://127.0.0.1:8000/openapi.json en development.

12.1 Uso de la consola Swagger

- 1 Iniciar backend local en puerto 8000.
- 2 Abrir <http://127.0.0.1:8000/docs>.
- 3 Presionar Login con Google si la consola está configurada con FIREBASE_WEB_.*.
- 4 Confirmar que el token se aplica al esquema HTTPBearer.
- 5 Ejecutar /me para validar identidad.
- 6 Probar /clientes, /dashboard, /interacciones, /oportunidades o /propuestas según rol del usuario.

13. Problemas comunes y acciones correctivas

Problema	Causa probable	Acción correctiva
uv no encontrado	uv no está instalado o no está en PATH.	Instalar uv, cerrar y abrir terminal, verificar <code>uv --version</code> .
Backend no inicia	Falta .env, FIREBASE_PROJECT_ID o credenciales Admin SDK.	Revisar Backend/.env, ruta de service account y variables Firebase.
Error de credenciales	GOOGLE_APPLICATION_CREDENTIALS apunta a archivo inexistente.	Usar ruta válida y no versionada; confirmar permisos de lectura.
Frontend no conecta	VITE_API_BASE_URL incorrecto o backend apagado.	Configurar http://127.0.0.1:8000 y levantar backend.
Login Google falla	Dominio local no autorizado en Firebase Auth.	Agregar localhost y 127.0.0.1 en dominios autorizados.
401 en API	Token ausente, expirado o mal enviado.	Reiniciar sesión y verificar header Authorization Bearer.
403 en API	Rol insuficiente, usuario inactivo o plataforma bloqueada.	Revisar documento users en Firestore: rol, activo, webApp/appMovil.
Puerto ocupado	Otro proceso usa 8000, 5173 o 5174.	Cerrar proceso o usar puerto alternativo en comando dev.
npm install falla	Versión de Node incompatible o caché corrupta.	Verificar Node 20+, eliminar node_modules y package-lock si procede bajo control técnico.
RAG no responde con proveedor	DEEPSEEK_API_KEY faltante o no hay contexto indexado.	Configurar variable backend, subir documentos o validar respuesta local sin contexto.

14. Criterio de instalación exitosa

La instalación local se considera exitosa cuando se cumplen todas las condiciones técnicas mínimas descritas en la siguiente matriz. El cumplimiento de esta matriz permite iniciar revisión

funcional, pruebas UAT locales o preparación de despliegue, pero no reemplaza la aprobación formal del ambiente productivo.

Código	Criterio	Evidencia esperada
IL-01	Dependencias instaladas	git, Python, uv, Node.js y npm responden por consola.
IL-02	Backend operativo	/health responde status ok en puerto 8000.
IL-03	Readiness disponible	/readiness entrega reporte no sensible.
IL-04	Frontend operativo	Login carga en puerto 5173.
IL-05	Firebase funcional	Login Google obtiene token válido.
IL-06	Usuario sincronizado	/auth/login crea o actualiza usuario en Firestore.
IL-07	Autorización backend	/me responde uid, email, rol y activo.
IL-08	Flujo de clientes	Usuario con rol asignado puede listar o crear cliente según permiso.
IL-09	Build/lint	npm run lint/build y pruebas backend sin errores críticos.
IL-10	Documentación API	/docs permite probar endpoints en modo development.

Cierre del procedimiento

Una instalación local válida debe permitir levantar backend, frontend web y app vendedor web/PWA, autenticar con Google, ejecutar endpoints protegidos y revisar al menos un flujo comercial básico. Cualquier defecto posterior debe clasificarse como problema de configuración local, bug UAT, dependencia de ambiente o cambio de alcance según su naturaleza.